

中山大学专业课《程序设计I》

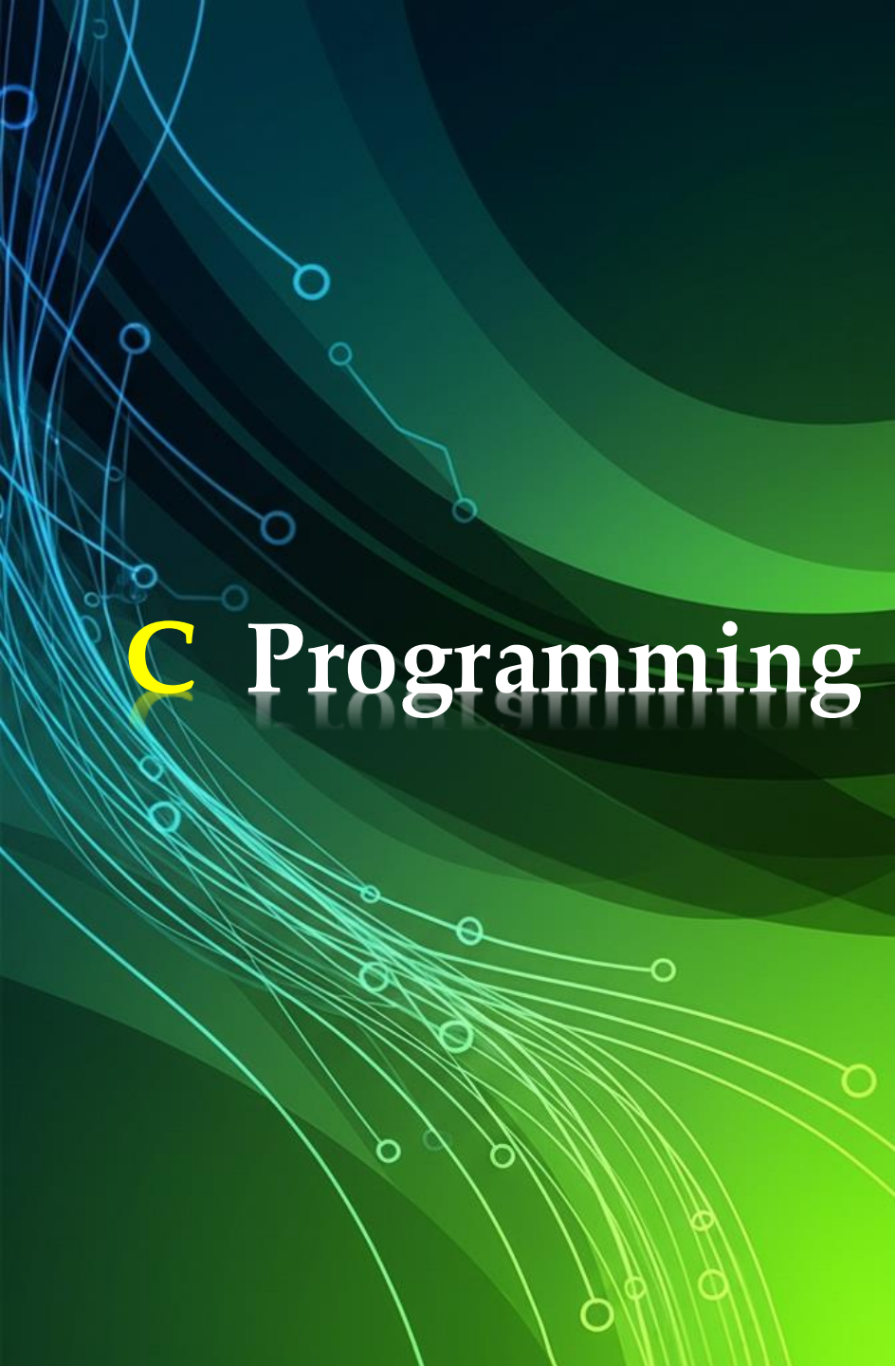
第十二节 链表

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季



C Programming

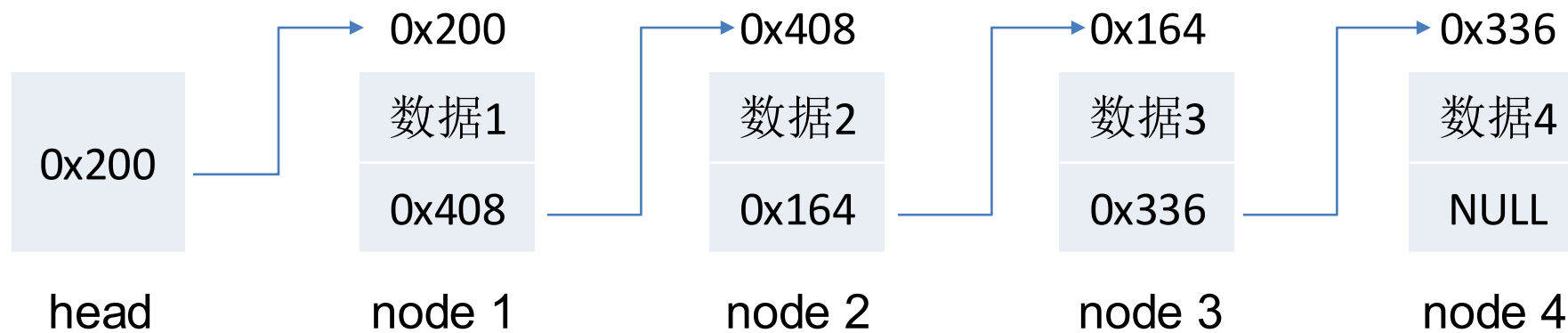
结构体

```
struct STUDENT {  
    char id[6];  
    char name[10];  
    int age;  
};
```

如何动态存储一系列结构体数据？

什么是链表 (Linked List)

- ❑ 链表是一种重要的**数据结构**，能够动态地进行存储分配

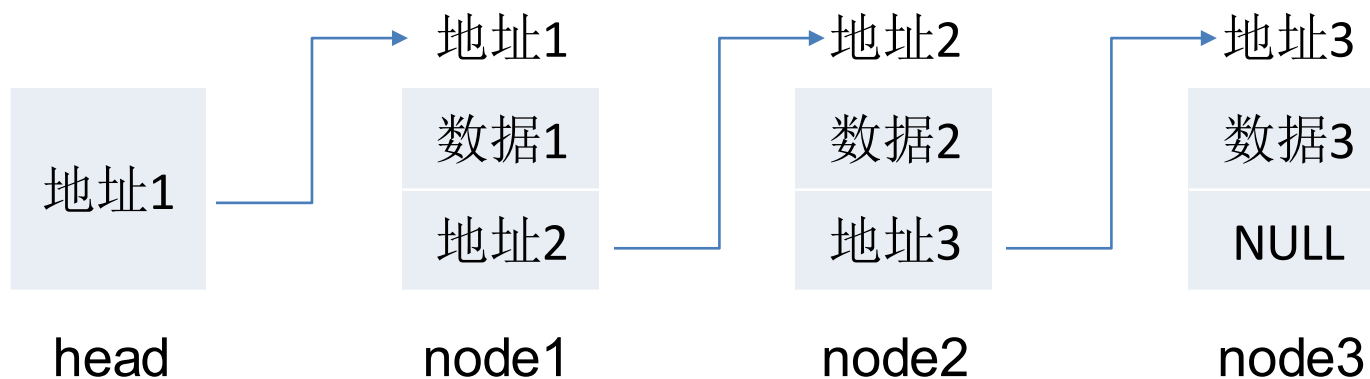


- ❑ 链表有一个**头指针**变量，图中以head表示，它存放链表第一个元素的地址
- ❑ 链表中每一个元素称为**结点 (node)**，包括两个部分：
 1. 用户需要用的实际数据
 2. 下一个结点的地址
- ❑ 最后一个元素称为**表尾 (tail)**，它的下一节点地址为NULL，表示链表结束

用自引用结构体实现链表

- ❑ 结构体不能嵌套自身类型，但可以通过成员变量引用自身类型
- ❑ 例如：struct NODE 中成员 next 指针引用 struct NODE

```
struct NODE {  
    int value;  
    struct NODE *next;  
};
```



用自引用结构体实现链表

```
typedef struct NODE Node;  
typedef Node *NODE_PTR;  
  
struct NODE {  
    int value;  
    NODE_PTR next;  
};
```

预声明结构体别名

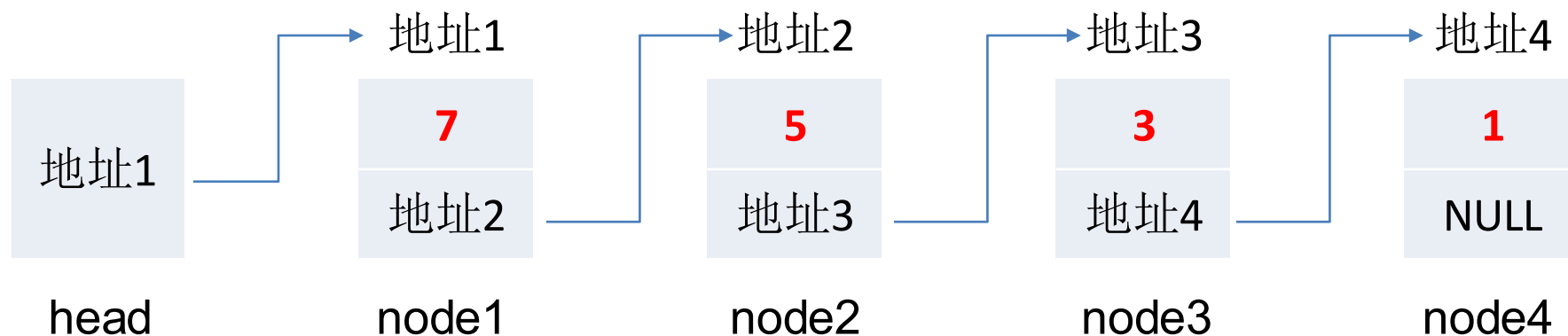
声明结构体指针类型

定义结构体类型

定义自引用成员

练习

创建该链表:



```
typedef struct NODE Node;
typedef Node *NODE_PTR;

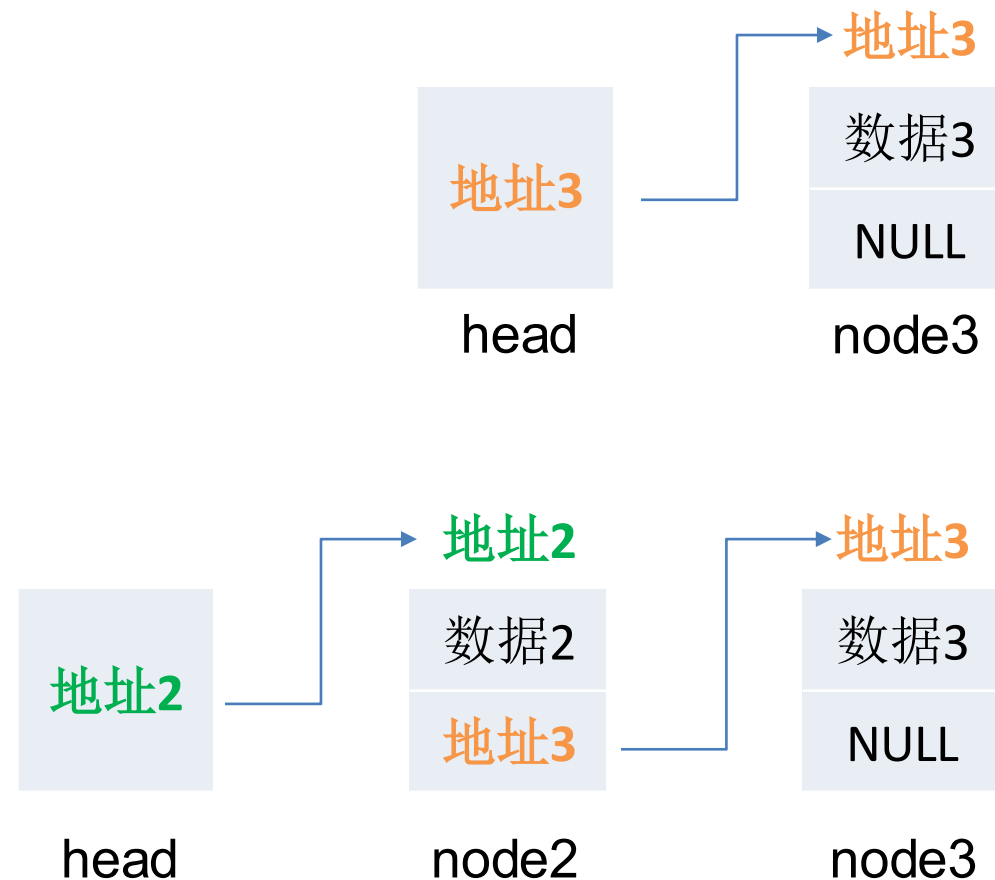
struct NODE {
    int value;
    NODE_PTR next;
};
```

链表基本操作 - 元素添加与移除

按链表定义可以在**链表头部**添加和移除元素

```
/*list ops*/
NODE_PTR list_add(NODE_PTR head, int i){
    NODE_PTR np = malloc(sizeof(Node));
    np->value = i;  np->next = head;
    return np;
}

NODE_PTR list_remove(NODE_PTR head){
    if (head) {
        NODE_PTR p = head->next;
        free(head);
        return p;
    }
    return NULL;
}
```



链表基本操作 - 创建与释放

按前述定义使用数组对进行创建或释放

```
/*list ops*/
NODE_PTR list_create(int *arr, size_t n) {
    NODE_PTR np = NULL;
    for(int i=n-1;i>-1;i--)
        np = list_add(np,arr[i]);
    return np;
}

NODE_PTR list_free(NODE_PTR head) {
    while (head)
        head = list_remove(head);
    return head;
}
```

链表基本操作 - 长度与索引

链表也可以和字符串一样**计算长度**或索引访问

```
/*list ops*/
size_t list_length(NODE_PTR li) {
    if (li) return list_length(li->next) + 1;
    else return 0;
}

NODE_PTR list_index(NODE_PTR li, size_t Idx) {
    if (li && Idx) return list_index(li->next, --Idx);
    else return li;
}
```

链表不能随机访问（等时间访问），只能顺序访问

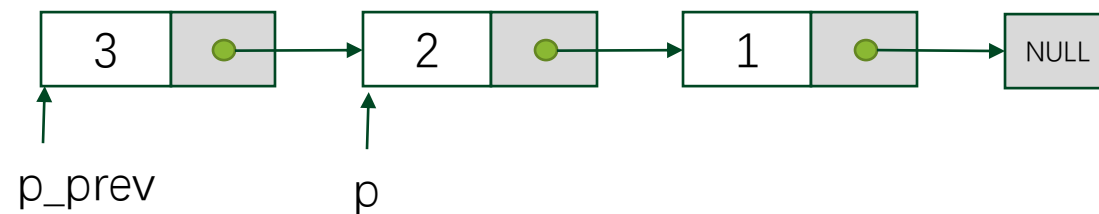
链表操作 - 指定位置元素插入与删除

如何在链表中删除指定位置元素或在指定位置插入元素？

显然，直接用当前指针移除子链表首元素导致链表断裂

```
/*ex-list*/
NODE_PTR list_delete(NODE_PTR *li){
    *li = list_remove(*li);
    return *li;
}

NODE_PTR list_insert(NODE_PTR *li, int i){
    ? 请完成
}
```



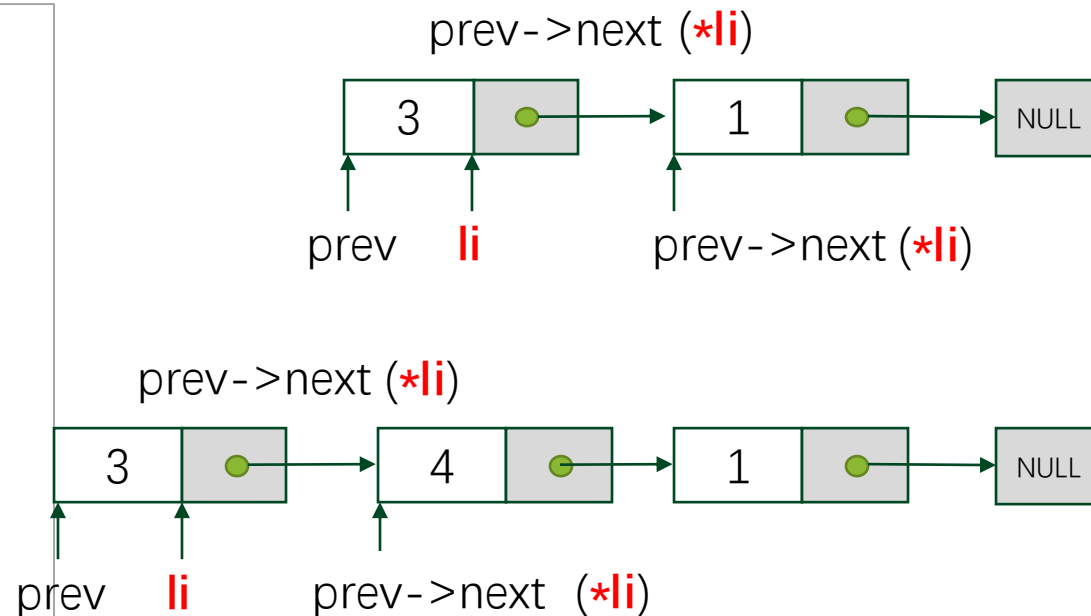
链表操作 - 指定位置元素插入与删除

如何在链表中删除指定位置元素或在指定位置插入元素?

显然，直接用当前指针移除子链表首元素导致链表断裂

```
/*ex-list*/
NODE_PTR list_delete(NODE_PTR *li){
    *li = list_remove(*li);
    return *li;
}

NODE_PTR list_insert(NODE_PTR *li, int i){
    *li = list_add(*li, i);
    return *li;
}
```



练习

1. 给出 `list_insert` 的函数描述及其实现代码
2. 给出 `list_insert` 和 `list_delete` 的测试代码
3. 完成链表元素查找和翻转链表操作
 - 链表元素查找。思考：链表元素查找可用二分法吗？
 - 翻转链表。要求：请给出两种或以上计算方法

谢谢!

邮箱: yuesh5@mail.sysu.edu.cn